

Лабораторная работа № 8.

Оптимизация

Абакумова Олеся Максимовна, НФИбд-02-22

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Предварительные сведения	8
3.1.1	Линейное программирование	8
3.1.2	Векторизованные ограничения и целевая функция оптимизации	11
3.1.3	Оптимизация рациона питания	14
3.1.4	Путешествие по миру	19
3.1.5	Портфельные инвестиции	21
3.1.6	Восстановление изображения	28
3.2	Задания для самостоятельного выполнения	32
4	Выводы	40

Список иллюстраций

3.1	Определение объекта модели и определение граничных усл. для переменных	10
3.2	Определение модели, цел. функции, ее вызов, статус работы оптимизатора и результат	11
3.3	Определение объекта модели, начальных данных и вектора переменных	13
3.4	Определение модели, функции, вызов статус работы оптимизатора и результат	14
3.5	Содержание калорий, белков, жиров и соли в продуктах питания .	15
3.6	Контейнер для хранения данных, массив данных с именами продуктов и массив стоимости продуктов	16
3.7	Массив данных о содержании КБЖУ, определение объекта модели и поределение массива с категориями	17
3.8	Определение переменных, целевой функции и ограничений модели	18
3.9	Вызов функции оптимизации и результат	18
3.10	Обозначения в сводной матрице по паспортам	19
3.11	Чтение файла, задание переменных и определение объекта модели	20
3.12	Переменные, ограничения и целевая функция. Вызов функции и результат	21
3.13	Подгружение данных	23
3.14	Изменение цен на акции компаний за 13 дней	23
3.15	Данные о ценах на акции в матрице, вычисление матрицы доходности, матрица рисков и проверка положительной определенности матрицы рисков	25
3.16	Доход от каждой из компаний, вектор инвестиций, объект модели	26
3.17	Нахождение решения	27
3.18	Вывод результатов	28
3.19	Исходное изображение и искусственно испорченное изображение в оттенках серого	30
3.20	Матрица цветов и решение	31
3.21	Норма и исправленное изображение	32
3.22	Решение задачи № 1	33
3.23	Решение задачи № 2	34
3.24	Решение задачи № 3	35
3.25	Решение задачи № 4	38
3.26	Решение задачи № 5	39

3.27 Решение задачи № 5	39
-----------------------------------	----

Список таблиц

1 Цель работы

Основная цель работы — освоить пакеты Julia для решения задач оптимизации.

2 Задание

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

3 Выполнение лабораторной работы

3.1 Предварительные сведения

Под оптимизацией в математике и информатике понимается решение задачи нахождения экстремума (минимума или максимума) целевой функции в некоторой области конечномерного векторного пространства, ограниченной набором линейных и/или нелинейных равенств и/или неравенств.

Оптимизационной задачей называется задача определения наилучших с точки зрения структуры или значений параметров объектов.

3.1.1 Линейное программирование

В Julia для обработки данных используются наработки из других языков программирования, в частности, из R и Python.

Линейное программирование рассматривает решения экстремальных задач на множествах n -мерного векторного пространства, задаваемых системами линейных уравнений и неравенств.

Общей (стандартной) задачей линейного программирования называется задача нахождения минимума линейной целевой функции вида:

$$f(\vec{x}) = \sum_{j=1}^n c_j x_j,$$

где $\vec{c}$ — коэффициенты, $\vec{x} \in \mathbb{R}^n$.

Основной задачей линейного программирования называется задача, в которой есть ограничения в форме неравенств:

$$\sum_{j=1}^n a_{ij}x_j \geq b_i, \quad i = 1, 2, \dots, m,$$

$$x_j \geq 0, \quad j = 1, 2, \dots, n.$$

Задачи линейного программирования со смешанными ограничениями, такими как равенства и неравенства, с наличием переменных, свободных от ограничений, могут быть сведены к эквивалентным с тем же множеством решений путём замены переменных и замены равенств на пару неравенств.

В Julia есть несколько средств, предназначенных для решения оптимизационных задач. Одним из таких средств является JuMP (<https://jump.dev/>) — язык моделирования и вспомогательные пакеты для формулирования и решения задач математической оптимизации в Julia.

JuMP включает пакет `Convex.jl` (<https://jump.dev/Convex.jl/stable/>), позволяющий описать задачу оптимизации, используя естественный математический синтаксис, и решать её с помощью одного из решателей (COSMO, ECOS, SCS, GLPK, MathOptInterface). Предположим, что требуется решить следующую задачу линейного программирования:

$$12x + 20y \rightarrow \min$$

при ограничениях:

$$6x + 8y \geq 100,$$

$$7x + 12y \geq 120,$$

$$x \geq 0, \quad y \geq 0.$$

Воспользуемся JuMP и решателем линейного и смешанного целочисленного программирования GLPK:

Подключение пакетов:

```
import Pkg
Pkg.add("JuMP")
Pkg.add("GLPK")
using JuMP
using GLPK
```

Объект модели (контейнер для переменных, ограничений, параметров решателя и т. д.) в JuMP создаётся при помощи функции `Model()`, в которой в качестве аргумента указывается оптимизатор (решатель).

Переменные задаются с помощью конструкции `@variable` (имя объекта модели, имя и привязка переменной, тип переменной). Здесь же задаются граничные условия на переменные (если тип переменной не определён, он считается действительным) (рис. 3.1):

```
]# Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}

]: A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

]: # Определение переменных x, y и граничных условий для них:
@variable(model, x >= 0)
@variable(model, y >= 0)
```

Рис. 3.1: Определение объекта модели и определение граничных усл. для переменных

В качестве первого аргумента указан объект модели `model`, затем переменные

х и у, связанные с этой моделью (причём указанные переменные не могут использоваться в другой модели).

Ограничения модели задаются с помощью конструкции `@constraint` (имя объекта модели, ограничение. Далее следует задать собственно целевую функцию с помощью конструкции `@objective` (имя объекта модели, Min или Max, функция для оптимизации). Для решения задачи оптимизации необходимо вызывать функцию оптимизации. Следует проверять причину прекращения работы оптимизатора, используя конструкцию `termination_status(Объект модели)`. Процесс решения мог быть прекращён по ряду причин. Во-первых, решатель мог найти оптимальное решение или доказать, что проблема невозможна. Однако он также мог столкнуться с численными трудностями или прерваться из-за таких настроек, как ограничение по времени. Если возвращено значение `OPTIMAL`, найдено оптимальное решение (рис. 3.2):

```
: # Определение ограничений модели:
@constraint(model, 6x + 8y >= 100)
@constraint(model, 7x + 12y >= 120)
: 7x + 12y ≥ 120

: # Определение целевой функции:
@objective(model, Min, 12x + 20y)
: 12x + 20y

: # Вызов функции оптимизации:
optimize!(model)

: # Определение причины завершения работы оптимизатора:
termination_status(model)
: OPTIMAL::TerminationStatusCode = 1

: # Демонстрация первичных результирующих значений переменных x и y:
@show value(x);
@show value(y);
: # Демонстрация результата оптимизации:
@show objective_value(model);
value(x) = 14.999999999999993
value(y) = 1.2500000000000004
objective_value(model) = 285.0
```

Рис. 3.2: Определение модели, цел. функции, ее вызов, статус работы оптимизатора и результат

3.1.2 Векторизованные ограничения и целевая функция оптимизации

Часто бывает полезно создавать коллекции переменных JuMP внутри более сложных структур данных. Можно добавить ограничения и цель в JuMP, исполь-

зую векторизован- ную линейную алгебру.

Предположим, что требуется решить следующую задачу:

$$c^T \vec{x} \rightarrow \min$$

при заданных ограничениях:

$$A\vec{x} = \vec{b}, \quad \vec{x} \geq 0, \quad \vec{x} \in \mathbb{R}^n,$$

где:

$$A = \begin{pmatrix} 1 & 1 & 9 & 5 \\ 3 & 5 & 0 & 8 \\ 2 & 0 & 6 & 13 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 7 \\ 3 \\ 5 \end{pmatrix}, \quad \vec{c} = \begin{pmatrix} 1 \\ 3 \\ 5 \\ 2 \end{pmatrix}.$$

Воспользуемся JuMP и решателем линейного и смешанного целочисленного програм- мирования GLPK:

Подключение пакетов:

```
import Pkg
Pkg.add("JuMP")
Pkg.add("GLPK")
using JuMP
using GLPK
```

Определим объект модели и зададим исходные значения матрицы A и векто- ров $\vec{b}$, $\vec{c}$. Далее зададим массив переменных для компонент вектора $\vec{x}$ (рис. 3.3):

```

: # Определение объекта модели с именем vector_model:
vector_model = Model(GLPK.Optimizer)

: A JuMP Model
  | solver: GLPK
  | objective_sense: FEASIBILITY_SENSE
  | num_variables: 0
  | num_constraints: 0
  | Names registered in the model: none

: # Определение начальных данных:
A= [ 1 1 9 5;
3 5 0 8;
2 0 6 13]
b = [7; 3; 5]
c = [1; 3; 5; 2]

: 4-element Vector{Int64}:
 1
 3
 5
 2

: # Определение вектора переменных:
@variable(vector_model, x[1:4] >= 0)

: 4-element Vector{VariableRef}:
 x[1]
 x[2]
 x[3]
 x[4]

```

Рис. 3.3: Определение объекта модели, начальных данных и вектора переменных

Затем зададим ограничения в соответствии с условиями модели, определим ограничений модели, целевую функцию, вызовем функцию оптимизации, проверим наличие оптимального решения и посмотрим результат решения оптимизационной задачи (рис. 3.4):

```

julia> # Определение ограничений модели:
@constraint(vector_model, A * x .== b)

julia> 3-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintInterface.EqualTo{Float64}}, ScalarShape{}}:
 x[1] + x[2] + 9 x[3] + 5 x[4] == 7
 3 x[1] + 5 x[2] + 8 x[4] == 3
 2 x[1] + 6 x[3] + 13 x[4] == 5

julia> # Определение целевой функции:
@objective(vector_model, Min, c' * x)

julia> x1 + 3x2 + 5x3 + 2x4

julia> # Вызов функции оптимизации:
optimize!(vector_model)

julia> # Определение причины завершения работы оптимизатора:
termination_status(vector_model)

julia> OPTIMAL::TerminationStatusCode = 1

julia> # Демонстрация результата оптимизации:
@show objective_value(vector_model);

objective_value(vector_model) = 4.9230769230769225

```

Рис. 3.4: Определение модели, функции, вызов статус работы оптимизатора и результат

3.1.3 Оптимизация рациона питания

В некоторых задачах требуется использование массивов, в которых индексы не являются целыми диапазонами с отсчётом от единицы. Например, требуется использовать переменную, индексируемую по названию продукта или местоположению. Тогда необходимо в качестве индекса использовать произвольный вектор. В Julia это можно реализовать с помощью `DenseAxisArrays`.

Рассмотрим применение `DenseAxisArrays` на примере решения задачи оптимизации рациона питания в заведении быстрого питания при условии, что задано ограничение на количество потребляемых калорий (1800–2200), белков (≥ 91), жиров (0–65) и соли (0–1779), а также перечень определённых продуктов питания с указанием их стоимости:

- гамбургер (2.49 ден.ед.),

- курица (2.89 ден.ед.),
- сосиска в тесте (1.50 ден.ед.),
- жареный картофель (1.89 ден.ед.),
- макароны (2.09 ден.ед.),
- пицца (1.99 ден.ед.),
- салат (2.49 ден.ед.),
- молочный коктейль (0.89 ден.ед.),
- мороженое (1.59 ден.ед.).

Также известно содержание калорий, белков, жиров и соли в указанных продуктах (рис. 3.5):

продукт	калории	белки	жиры	соль
гамбургер	10	24	26	730
курица	420	32	10	1190
сосиска в тесте	560	20	32	1800
жареный картофель	380	4	19	270
макароны	320	12	10	930
пицца	320	15	12	820
салат	320	31	12	1230
молочный коктейль	100	8	2.5	125
мороженное	330	8	10	180

Рис. 3.5: Содержание калорий, белков, жиров и соли в продуктах питания

Воспользуемся JuMP и решателем линейного и смешанного целочисленного программирования GLPK :

Подключение пакетов:

```
import Pkg
Pkg.add("JuMP")
```

```
Pkg.add("GLPK")
```

```
using JuMP
```

```
using GLPK
```

Создадим контейнер JuMP для хранения информации об ограничениях (минимальное, максимальное) на количество потребляемых калорий, белков, жиров и соли. Введём массив данных с наименованиями продуктов и стоимости продуктов: (рис. 3.6):

```
: # Контейнер для хранения данных об ограничениях на количество потребляемых калорий, белков, жиров и соли:
category_data = JuMP.Containers.DenseAxisArray{
  [1800 2200;
   91 Inf;
   0 65;
   0 1779],
  ["calories", "protein", "fat", "sodium"],
  ["min", "max"]}

: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["calories", "protein", "fat", "sodium"]
  Dimension 2, ["min", "max"]
And data, a 4x2 Matrix{Float64}:
1800.0 2200.0
 91.0   Inf
  0.0   65.0
  0.0 1779.0

: # массив данных с наименованиями продуктов:
foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]

: 9-element Vector{String}:
 "hamburger"
 "chicken"
 "hot dog"
 "fries"
 "macaroni"
 "pizza"
 "salad"
 "milk"
 "ice cream"

: # Массив стоимости продуктов:
cost = JuMP.Containers.DenseAxisArray{
  [2.49, 2.89, 1.50, 1.89, 2.09, 1.99, 2.49, 0.89, 1.59],
  foods)

: 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{Float64}:
 2.49
 2.89
 1.5
 1.89
 2.09
 1.99
 2.49
 0.89
 1.59
```

Рис. 3.6: Контейнер для хранения данных, массив данных с именами продуктов и массив стоимости продуктов

Введём сведения о содержании калорий, белков, жиров и соли в продуктах питания, определим объект модели, массив (рис. 3.7):


```

: # Массив данных о содержании калорий, белков, жиров и соли в продуктах питания:
food_data = JuMP.Containers.DenseAxisArray{
[410 24 26 730;
420 32 10 1190;
560 20 32 1800;
380 4 19 270;
320 12 10 930;
320 15 12 820;
320 31 12 1230;
100 8 2.5 125;
330 8 10 180],
foods,
["calories", "protein", "fat", "sodium"])

: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
  Dimension 2, ["calories", "protein", "fat", "sodium"]
And data, a 9x4 Matrix{Float64}:
410.0  24.0  26.0  730.0
420.0  32.0  10.0  1190.0
560.0  20.0  32.0  1800.0
380.0   4.0  19.0  270.0
320.0  12.0  10.0  930.0
320.0  15.0  12.0  820.0
320.0  31.0  12.0  1230.0
100.0   8.0   2.5  125.0
330.0   8.0  10.0  180.0

: # Определение объекта модели с именем model:
model = Model(GLPK.Optimizer)

: A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

: # Определим массив:
categories = ["calories", "protein", "fat", "sodium"]

: 4-element Vector{String}:
"calories"
"protein"
"fat"
"sodium"

```

Ак
Чтс

Рис. 3.7: Массив данных о содержании КБЖУ, определение объекта модели и определение массива с категориями

Далее зададим переменные, зададим целевую функцию минимизации цены, ограничения в соответствии с условиями модели (рис. 3.8):

```

]: # Определение переменных:
@variables(model, begin
category_data[c, "min"] <= nutrition[c - categories] <= category_data[c, "max"]
# Сколько покупать продуктов:
buy[foods] >= 0
end)

]: (1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{VariableRef}:
nutrition[calories]
nutrition[protein]
nutrition[fat]
nutrition[sodium], 1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{VariableRef}:
buy[hamburger]
buy[chicken]
buy[hot dog]
buy[fries]
buy[macaroni]
buy[pizza]
buy[salad]
buy[milk]
buy[ice cream])

]: # Определение целевой функции:
@objective(model, Min, sum(cost[f] * buy[f] for f in foods))

]: 2.49buy_hamburger + 2.89buy_chicken + 1.5buy_hotdog + 1.89buy_fries + 2.09buy_macaroni + 1.99buy_pizza + 2.49buy_salad + 0.89buy_milk + 1.59buy_icecream

]: # Определение ограничений модели:
@constraint(model, [c in categories],
sum(food_data[f, c] * buy[f] for f in foods) == nutrition[c])

]: 1-dimensional DenseAxisArray{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape{1},...} with index sets:
Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape{1}}}:
-nutrition[calories] + 410 buy[hamburger] + 420 buy[chicken] + 560 buy[hot dog] + 380 buy[fries] + 320 buy[macaroni] + 320 buy[pizza] + 320 buy[salad] + 100 buy[milk] + 330 buy[ice cream] == 0
-nutrition[protein] + 24 buy[hamburger] + 32 buy[chicken] + 20 buy[hot dog] + 4 buy[fries] + 12 buy[macaroni] + 15 buy[pizza]

```

Рис. 3.8: Определение переменных, целевой функции и ограничений модели

Для решения задачи оптимизации вызовем функцию оптимизации, для просмотра результата решения можно вывести значение переменной buy (рис. 3.9):

```

# Вызов функции оптимизации:
JuMP.optimize!(model)
term_status = JuMP.termination_status(model)

```

```

OPTIMAL::TerminationStatusCode = 1

```

```

hcat(buy.data, JuMP.value.(buy.data))

```

```

9x2 Matrix{AffExpr}:
 buy[hamburger]  0.6045138888888888
 buy[chicken]    0
 buy[hot dog]    0
 buy[fries]      0
 buy[macaroni]   0
 buy[pizza]      0
 buy[salad]      0
 buy[milk]       6.9701388888888935
 buy[ice cream]  2.5913194444444441

```

Рис. 3.9: Вызов функции оптимизации и результат

В результате оптимальным по цене и пищевой ценности будет предложено купить гамбургер, молочный коктейль и мороженное.

3.1.4 Путешествие по миру

Рассмотрим решение задачи определения оптимального числа паспортов, требующихся, чтобы объехать весь мир. При этом будем учитывать сведения о паспортах и ограничениях, указанных на ресурсе <https://www.passportindex.org/>. В табличной форме данные можно получить с ресурса <https://github.com/ilyankou/passport-index-dataset>. Для решения задачи представляют интерес данные, указанные в файле `passport-index-matrix.csv`, в котором в первом столбце (Passport) указана страна отбытия (= от), в остальных столбцах указаны страны назначения (= до), на пересечении строк и столбцов указаны условия по наличию или отсутствию визы или другие ограничения (рис. 3.10):

Значение	Пояснение
7–360	Количество безвизовых дней (при наличии)
VF	viza free (безвизовый режим)
VOA	viza on arriva (виза по прибытии)
ETA	e-visa или electronic travel authority (электронная виза)
VR	viza required (требуется виза)
covid ban	запрет в связи с COVID=19
no admission	въезд запрещён
-1	паспорт из места назначения

Рис. 3.10: Обозначения в сводной матрице по паспортам

Итак, попробуем решить задачу средствами Julia:

```
## Подключение пакетов:  
import Pkg  
Pkg.add("DelimitedFiles")  
Pkg.add("CSV")  
using DelimitedFiles  
using CSV
```

Можем считать данные из имеющегося файла. Далее просматриваем файл, задаём переменную для подсчёта числа паспортов, задаём переменную vf, в которую заносим сведения при отсутствии необходимости получать визу (если в поле указано число, «VF» или «VOA») и определяем объект модели (рис. 3.11):

```
passportdata = readelm(joinpath("passport-index-dataset", "C:/Users/user/Downloads/passport-index-matrix.csv"), ',', ' ')

200x200 Matrix{Any}:
"Passport"      "Albania"  ...  "Afghanistan"
"Afghanistan"    "e-visa"   -1
"Albania"        -1          "visa required"
"Algeria"        "e-visa"   "visa required"
"Andorra"        90          "visa required"
"Angola"         "e-visa"   ...  "visa required"
"Antigua and Barbuda" 90          "visa required"
"Argentina"      90          "visa required"
"Armenia"        90          "visa required"
"Australia"      90          "visa required"
"Austria"        90          ...  "visa required"
"Azerbaijan"     90          "visa required"
"Bahamas"        90          "visa required"
⋮
"United Arab Emirates" 90          "visa required"
"United Kingdom"      90          "visa required"
"United States"       360         ...  "visa required"
"Uruguay"            90          "visa required"
"Uzbekistan"         "e-visa"   "visa required"
"Vanuatu"            "e-visa"   "visa required"
"Vatican"           90          "visa required"
"Venezuela"         90          ...  "visa required"
"Vietnam"           "e-visa"   "visa required"
"Yemen"             "e-visa"   "visa required"
"Zambia"            "e-visa"   "visa required"
"Zimbabwe"          "e-visa"   "visa required"

# Задаём переменные:
cntr = passportdata[2:end,1]
vf = (x -> typeof(x) == Int64 || x == "VF" || x == "VOA" ? 1 : 0).(passportdata[2:end,2:end]);

# Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none
```

Ак
Чтс

Рис. 3.11: Чтение файла, задание переменных и определение объекта модели

Добавляем переменные, ограничения и целевую функцию. Для решения задачи оптимизации вызываем функцию оптимизации, просматриваем результат (рис. 3.12):

```

# Переменные, ограничения и целевая функция:
@variable(model, pass[1:length(cntr)], Bin)
@constraint(model, [j=1:length(cntr)], sum( vf[i,j]*pass[i] for i in 1:length(cntr)) >= 1)
@objective(model, Min, sum(pass))

pass1 + pass2 + pass3 + pass4 + pass5 + pass6 + pass7 + pass8 + pass9 + pass10 + pass11 + pass12 + pass13 + pass14 + pass15 + pass16
+ pass17 + pass18 + pass19 + pass20 + pass21 + pass22 + pass23 + pass24 + pass25 + pass26 + pass27 + pass28 + pass29 + pass30
+ [... 139 terms omitted ...] + pass170 + pass171 + pass172 + pass173 + pass174 + pass175 + pass176 + pass177 + pass178 + pass179 + pass180
+ pass181 + pass182 + pass183 + pass184 + pass185 + pass186 + pass187 + pass188 + pass189 + pass190 + pass191 + pass192 + pass193
+ pass194 + pass195 + pass196 + pass197 + pass198 + pass199

# Вызов функции оптимизации:
JuMP.optimize!(model)
termination_status(model)

OPTIMAL::TerminationStatusCode = 1

# Просмотр результата:
print(JuMP.objective_value(model), " passports: ", join(cntr[findall(JuMP.value.(pass) .== 1)], ", "))

34.0 passports: Afghanistan, Australia, Bahrain, Cameroon, Comoros, Congo, Djibouti, Dominican Republic, Eritrea, Guinea-Bissau, Hong Kong, Iran, Kenya, Kuwait, Liberia, Libya, Madagascar, Maldives, Mauritania, Morocco, Nepal, New Zealand, North Korea, Palestine, Papua New Guinea, Qatar, Saudi Arabia, Singapore, Somalia, South Sudan, Spain, Syria, Turkmenistan, United States

```

Рис. 3.12: Переменные, ограничения и целевая функция. Вызов функции и результат

3.1.5 Портфельные инвестиции

Портфельные инвестиции — размещение капитала в ценные бумаги, формируемые в виде портфеля ценных бумаг, с целью получения прибыли.

Инвестиционный портфель — процесс стратегического управления капиталом как оптимизированной, единой системой инвестиционных ценностей.

Предположим требуется решить оптимизационную задачу в следующей формулировке. Имеется капитал в 1000 ден. ед., который планируется инвестировать в три компании — Microsoft, Facebook, Apple. При этом есть данные еженедельных значений цен на акции этих компаний за определённый период времени. Необходимо определить доходность акций каждой компании за рассматриваемый период времени, после чего инвестировать в эти три компании так, чтобы получить возврат в размере не менее 2% от вложенной суммы.

Для решения оптимизационной задачи будем использовать пакет `Convex.jl` и оптимизатор (решатель) `SCS`. Кроме того, понадобится пакет `Statistics.jl` для получения матрицы рисков на основе расчёта ковариационных значений по доходности.

Сначала требуется подгрузить имеющиеся данные о еженедельных значениях цен на акции рассматриваемых компаний, отобразить данные на графике. Предположим данные размещены в файле `stock_prices.xlsx` в каталоге `data` в

проекте.

Подключаем пакеты:

Подключение необходимых пакетов:

```
import Pkg
Pkg.add("DataFrames")
Pkg.add("XLSX")
Pkg.add("Plots")
Pkg.add("PyPlot")
Pkg.add("Convex")
Pkg.add("SCS")
Pkg.add("Statistics")
using DataFrames
using XLSX
using Plots
pyplot()
using Convex
using SCS
using Statistics
```

Подгружаем данные еженедельных значений цен на акции компаний за определённый период времени во фрейм (рис. 3.13):

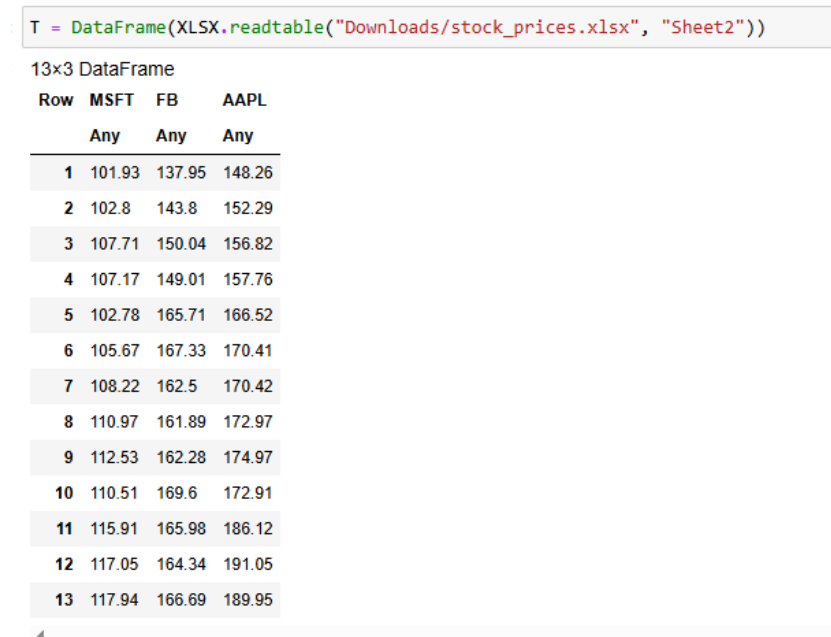


Рис. 3.13: Подгружение данных

Отображаем данные на графике (рис. 3.14):

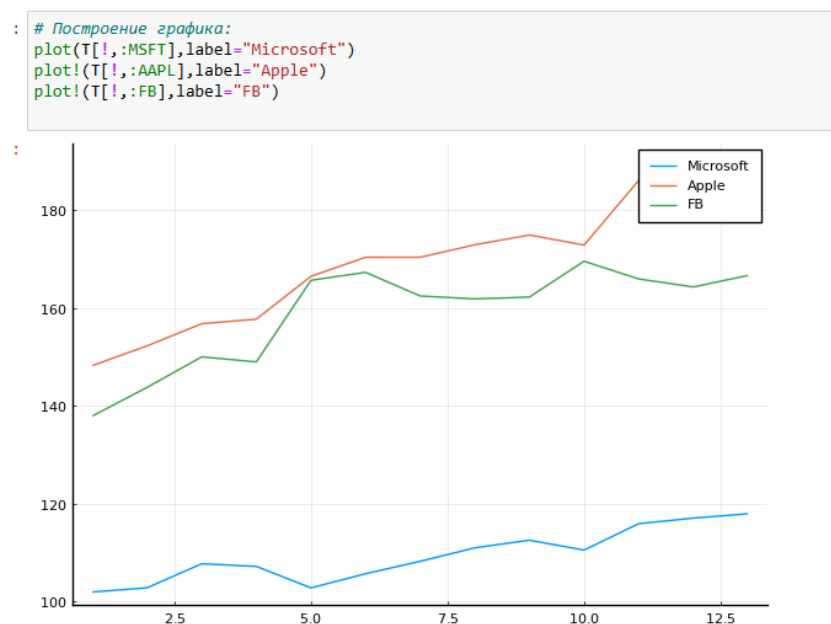


Рис. 3.14: Изменение цен на акции компаний за 13 дней

Для дальнейших расчётов данные по ценам на акции переформируем из фрейма в матрицу. Следует опираться на следующие формулы:

1. Доходность акций:

$$R(i, t) = \frac{pr(i, t) - pr(i, t - 1)}{pr(i, t - 1)}.$$

2. Целевая функция (минимизация риска):

$$\vec{x}^T \cdot \text{cov}(R) \cdot \vec{x} \rightarrow \min .$$

3. Ограничения:

$$\sum_{i=1}^3 x_i = 1,$$

$$\text{dot}(\vec{r}, \vec{x}) \geq 0.02,$$

$$x_i \geq 0, \quad i = 1, 2, 3.$$

Далее необходимо сформировать матрицу рисков — ковариационную матрицу рассчитанных цен доходности (рис. 3.15):


```

: # Данные о ценах на акции размещаем в матрице:
prices_matrix = Matrix{T}

: 13x3 Matrix{Any}:
101.93  137.95  148.26
102.8   143.8   152.29
107.71  150.04  156.82
107.17  149.01  157.76
102.78  165.71  166.52
105.67  167.33  170.41
108.22  162.5   170.42
110.97  161.89  172.97
112.53  162.28  174.97
110.51  169.6   172.91
115.91  165.98  186.12
117.05  164.34  191.05
117.94  166.69  189.95

: # Вычисление матрицы доходности за период времени:
M1 = prices_matrix[1:end-1,:]
M2 = prices_matrix[2:end,:]
# Матрица доходности:
R = (M2.-M1)./M1

: 12x3 Matrix{Float64}:
 0.00853527  0.0424067  0.027182
 0.0477626  0.0433936  0.0297459
-0.00501346 -0.00686484  0.00599413
-0.040963   0.112073   0.0555274
 0.0281183   0.00977611  0.0233606
 0.0241317  -0.0288651   5.8682e-5
 0.0254112  -0.00375385  0.014963
 0.0140579  0.00240904  0.0115627
-0.0179508  0.0451072  -0.0117734
 0.0488644  -0.0213443  0.0763981
 0.00983522 -0.00988071  0.0264883
 0.00760359  0.0142996 -0.00575766

: # Матрица рисков:
risk_matrix = cov(R)
# Проверка положительной определённости матрицы рисков:
isposdef(risk_matrix)

: true

```

Рис. 3.15: Данные о ценах на акции в матрице, вычисление матрицы доходности, матрица рисков и проверка положительной определённости матрицы рисков

Доход от каждой из компаний получим из матрицы доходности как вектор средних значений. Далее нужно собственно сформулировать оптимизационную задачу. Пусть вектор $\vec{x} = (x_1, x_2, x_3)$ — вектор инвестиций, вектор

$\vec{r} = (r_1, r_2, r_3)$ — вектор доходов от компаний. Тогда задача оптимизации будет иметь вид:

$$\vec{x}^T \text{cov}(R) \vec{x} \rightarrow \min$$

при условии:

$$\sum_{i=1}^3 x_i = 1, \quad \text{dot}(\vec{r}, \vec{x}) \geq 0,02, \quad x_i \geq 0, \quad i = 1, 2, 3,$$

где $\text{dot}(\vec{r}, \vec{x})$ — функция, определяющая возврат инвестиций. Формируем вектор инвестиций и определяем объект модели в соответствии с формулировкой оптимизационной задачи (при этом делаем задачу совместимой с DCP в соответствии с требованием пакета `Convex.jl` (рис. 3.16):

```
: # Доход от каждой из компаний:
r = mean(R,dims=1)[: ]

: 3-element Vector{Float64}:
 0.012532748705136572
 0.016563036855293173
 0.02114580465503291

: # Вектор инвестиций:
x = Variable(length(r))

: Variable
size: (3, 1)
sign: real
vexity: affine
id: 385...610

: # Объект модели:
problem = minimize(Convex.quadform(x,risk_matrix),[sum(x)==1;r'*x>=0.02;x.>=0])

: Problem statistics
  problem is DCP           : true
  number of variables      : 1 (3 scalar elements)
  number of constraints     : 5 (5 scalar elements)
  number of coefficients   : 19
  number of atoms          : 14

Solution summary
  termination status : OPTIMIZE_NOT_CALLED
  primal status      : NO_SOLUTION
  dual status        : NO_SOLUTION
```

Рис. 3.16: Доход от каждой из компаний, вектор инвестиций, объект модели

Решаем поставленную задачу (рис. 3.17):

```

: # Находим решение:
  solve!(problem, SCS.Optimizer)

-----
          SCS v3.2.9 - Splitting Conic Solver
        (c) Brendan O'Donoghue, Stanford University, 2012
-----
problem:  variables n: 5, constraints m: 12
cones:    z: primal zero / dual free vars: 1
          l: linear vars: 4
          q: soc vars: 7, qsize: 2
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys:  sparse-direct-amd-qdldl
          nnz(A): 22, nnz(P): 0
-----

iter | pri res | dua res | gap | obj | scale | time (s)
-----
  0 | 1.45e+01 | 1.00e+00 | 2.01e+01 | -9.96e+00 | 1.00e-01 | 2.97e-03
 75 | 1.70e-04 | 3.73e-05 | 5.25e-06 | 4.62e-04 | 1.00e-01 | 3.35e-03
-----

status:  solved
timings: total: 3.36e-03s = setup: 2.23e-04s + solve: 3.13e-03s
          lin-sys: 3.35e-05s, cones: 3.17e-05s, accel: 4.90e-06s
-----
objective = 0.000462
-----

[ Info: [Convex.jl] Compilation finished: 13.07 seconds, 916.313 MiB of m

: Problem statistics
  problem is DCP      : true
  number of variables : 1 (3 scalar elements)
  number of constraints: 5 (5 scalar elements)
  number of coefficients: 19
  number of atoms      : 14

Solution summary
  termination status : OPTIMAL
  primal status      : FEASIBLE_POINT
  dual status        : FEASIBLE_POINT
  objective value     : 0.0005

```

Рис. 3.17: Нахождение решения

Выводим значения компонент вектора инвестиций и проверяем выполнение условия $\sum_{i=1}^3 x_i = 1$. Также проверяем выполнение условия на уровень доходности от 2%, переводим процентные значения компонент вектора инвестиций в фактические денежные единицы. Далее сформируем новый фрейм, включающий исходные данные о недвижимости и столбец с данными о назначенном каждому дому кластере (рис. 3.18):

```

: x
: Variable
  size: (3, 1)
  sign: real
  vexity: affine
  id: 385...610
  value: [0.07740709795031048, 0.11606771003983767, 0.8065276266877964]

: sum(x.value)
: 1.0000024346779446

: r'*x.value
: 1x1 adjoint(::Vector{Float64}) with eltype Float64:
  0.0199472331085319

: x.value .* 1000
: 3x1 Matrix{Float64}:
  77.40709795031047
  116.06771003983766
  806.5276266877964

```

Рис. 3.18: Вывод результатов

В результате, получаем: надо инвестировать 77.4 ден. ед. в Microsoft, 116.1 ден. ед. в Facebook, 809.5 ден. ед. в Apple.

3.1.6 Восстановление изображения

Предположим есть изображение, на котором были изменены некоторые пиксели. Требуется восстановить неизвестные пиксели путём решения задачи оптимизации. Будем использовать пакет `Convex.jl` и оптимизатор (решатель) `SCS`, пакет `ImageMagick.jl` для работы с изображениями:

Подключение необходимых пакетов:

```

import Pkg
Pkg.add("ImageMagick")
Pkg.add("Convex")
Pkg.add("SCS")
using Images

```

```
using Convex
```

```
using SCS
```

Загрузим изображение для последующей обработки, преобразуем изображение в оттенки серого и испортим некоторые пиксели(рис. 3.19):

```
# Считывание исходного изображения:  
Kref = load("Downloads/khiam-small.jpg")
```



```
K = copy(Kref)  
p = prod(size(K))  
missingids = rand(1:p,400)  
K[missingids] .= RGBX{N0f8}(0.0,0.0,0.0)  
K  
Gray.(K)
```

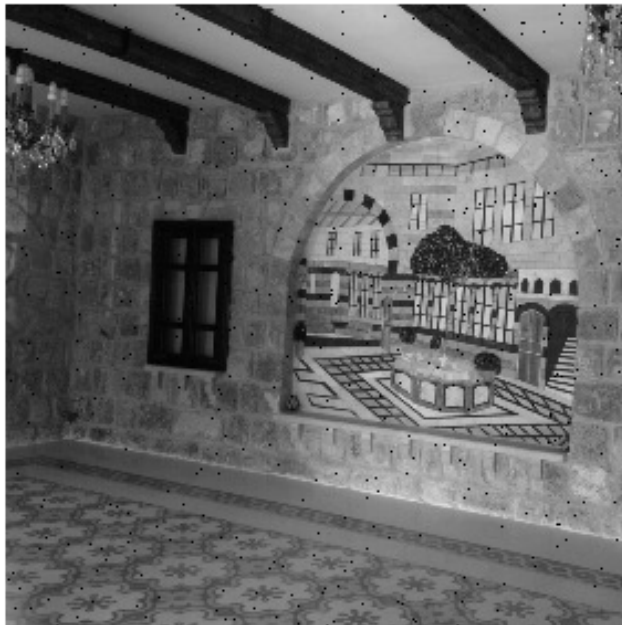


Рис. 3.19: Исходное изображение и искусственно испорченное изображение в оттенках серого

Формируем матрицу со значениями цветов, далее необходимо сформировать новую матрицу X , в которой минимизируется норма ядра матрицы (т.е. сумма сингулярных чисел элементов матрицы) так, что элементы, которые уже известны в матрице Y , остаются теми же самыми в матрице X . Решаем поставленную задачу (рис. 3.20):

```
# Матрица цветов:
Y = Float64.(Gray.(K));

correctids = findall(Y[:,!0])
X = Convex.Variable(size(Y))
problem = minimize(nuclearnorm(X))
problem.constraints += X[correctids]==Y[correctids]

1-element Vector{Constraint}:
 == constraint (affine)
└─ + (affine; real)
    └─ index (affine; real)
        └─ 283x283 real variable (id: 127...095)
            └─ 79690x1 Matrix{Float64}

solve!(problem, SCS.Optimizer)
```

```
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012

problem: variables n: 80090, constraints m: 159780
cones:   z: primal zero / dual free vars: 79690
         nuc: nuclear vars: 80090, nucsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys: sparse-direct-amd-qdldl
          nnz(A): 159780, nnz(P): 0

-----
iter | pri res | dua res | gap | obj | scale | time (s)
-----
0 | 1.27e+02 | 1.17e+01 | 3.09e+03 | 1.53e+03 | 1.00e-01 | 3.89e-01
150 | 4.32e-03 | 6.54e-05 | 1.21e-04 | 4.45e+02 | 9.31e-03 | 1.57e+01
-----

status: solved
timings: total: 1.57e+01s = setup: 2.29e-01s + solve: 1.55e+01s
         lin-sys: 7.86e-01s, cones: 1.43e+01s, accel: 6.20e-02s
```

Рис. 3.20: Матрица цветов и решение

Выводим значение нормы и исправленное изображение (рис. 3.21):

```

: @show norm(float.(Gray.(Kref))-X.value)
  @show norm(-(X.value))
  colorview(Gray, X.value)

norm(float.(Gray.(Kref)) - X.value) = 1.129468725543763
norm(-(X.value)) = 124.33326959946373

```



Рис. 3.21: Норма и исправленное изображение

3.2 Задания для самостоятельного выполнения

1. Решите задачу линейного программирования (рис. 3.22):

$$x_1 + 2x_2 + 5x_3 \rightarrow \max,$$

при заданных ограничениях:

$$-x_1 + x_2 + 3x_3 \leq -5, \quad x_1 + 3x_2 - 7x_3 \leq 10, \quad 0 \leq x_1 \leq 10, \quad x_2 \geq 0, \quad x_3 \geq 0.$$


```

: # 1
  # Создание модели
  model = Model(GLPK.Optimizer)

  # Определение переменных
  @variable(model, 0 <= x1 <= 10)
  @variable(model, x2 >= 0)
  @variable(model, x3 >= 0)

  # Определение ограничений
  @constraint(model, -x1 + x2 + 3x3 <= -5)
  @constraint(model, x1 + 3x2 - 7x3 <= 10)

  # Определение целевой функции (максимизация)
  @objective(model, Max, x1 + 2x2 + 5x3)

  # Решение задачи
  optimize!(model)

  # Вывод результатов
  println("Статус решения: ", termination_status(model))
  println("Оптимальное значение целевой функции: ", objective_value(model))
  println("Оптимальные значения переменных:")
  println("x1 = ", value(x1))
  println("x2 = ", value(x2))
  println("x3 = ", value(x3))

  Статус решения: OPTIMAL
  Оптимальное значение целевой функции: 19.0625
  Оптимальные значения переменных:
  x1 = 10.0
  x2 = 2.1875
  x3 = 0.9375

```

Рис. 3.22: Решение задачи № 1

2. Решите предыдущее задание, используя массивы вместо скалярных переменных (рис. 3.23).

Рекомендация. Запишите систему ограничений в виде $A\vec{x} = \vec{b}$, а целевую функцию как $c^T\vec{x}$.

```

# 2
# Определение данных
c = [1, 2, 5] # Коэффициенты целевой функции
A = [-1 1 3; # Матрица ограничений
      1 3 -7]
b = [-5, 10] # Правые части ограничений
lb = [0, 0, 0] # Нижние границы
ub = [10, Inf, Inf] # Верхние границы

# Создание модели
model = Model(GLPK.Optimizer)

# Определение переменных с границами
@variable(model, x[1:3])
for i in 1:3
    set_lower_bound(x[i], lb[i])
    if ub[i] < Inf
        set_upper_bound(x[i], ub[i])
    end
end

# Определение ограничений
@constraint(model, constraints[j in 1:2], sum(A[j, i] * x[i] for i in 1:3) <= b[j])

# Определение целевой функции
@objective(model, Max, sum(c[i] * x[i] for i in 1:3))

# Решение задачи
optimize!(model)

# Вывод результатов
println("Статус решения: ", termination_status(model))
println("Оптимальное значение целевой функции: ", objective_value(model))
println("Оптимальные значения переменных:")
for i in 1:3
    println("x$i = ", value(x[i]))
end

Статус решения: OPTIMAL
Оптимальное значение целевой функции: 19.0625
Оптимальные значения переменных:
x1 = 10.0
x2 = 2.1875
x3 = 0.9375

```

Рис. 3.23: Решение задачи № 2

3. Решите задачу оптимизации (рис. 3.24):

$$|A\vec{x} - \vec{b}|_2^2 \rightarrow \min$$

при заданных ограничениях:

$$\vec{x} \geq 0,$$

где $\vec{x} \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $\vec{b} \in \mathbb{R}^m$.

Матрицу A и вектор $\vec{b}$ задайте случайным образом. Для решения задачи используйте пакет Convex и решатель SCS.

```

: # 3
# задание размеров
m = 10
n = 5

# Генерация случайной матрицы A и вектора b
# Для воспроизводимости можно установить seed через Random, но Convex не требует этого
A = randn(m, n)
b = randn(m)

# Определение переменной оптимизации (вектор длины n)
x = Variable(n)

# Целевая функция: минимизация квадрата евклидовой нормы
objective = sumsquares(A * x - b)

# Ограничение: x >= 0 (все компоненты неотрицательны)
constraints = [x >= 0]

# Формулировка задачи
problem = minimize(objective, constraints)

# Решение задачи с использованием SCS
solve!(problem, SCS.Optimizer; silent_solver = true)

# Вывод результата
println("Оптимальное значение x:")
println(x.value)

println("Минимальное значение целевой функции:")
println(problem.optval)

Оптимальное значение x:
[0.025014096084161812; 0.21866317249002198; 0.00012957944875598222; 7.203721200778569e-5; 6.208978223907359e-5;;]
Минимальное значение целевой функции:
8.010200224053998

```

Рис. 3.24: Решение задачи № 3

4. Проводится конференция с 5 разными секциями. Забронировано 5 залов различной вместимости: в каждом зале не должно быть меньше 180 и больше 250 человек, а на третьей секции активность подразумевает, что должно быть точно 220 человек. В заявке участник указывает приоритет посещения секции: 1 — максимальный приоритет, 3 — минимальный, а значение 10000 означает, что человек не пойдёт на эту секцию. Организаторам удалось собрать 1000 заявок с указанием приоритета посещения трёх секций. Необходимо дать рекомендацию слушателю, на какую же секцию ему пойти, чтобы хватило места всем. Для решения задачи используйте пакет Convex и решатель GLPK. Приоритеты по слушателям распределите случайным образом (рис. 3.25).

Решение:

```

# 4
# Настройка генератора случайных чисел
Random.seed!(123)

```

```

# Параметры задачи
num_sections = 5
num_participants = 1000
hall_capacities = [180, 190, 220, 200, 210] # Вместимость залов
hall_max = fill(250, num_sections) # Максимальная вместимость

# Генерация случайных приоритетов
# 1 - максимальный приоритет, 3 - минимальный, 10000 - не пойдет
priorities = Array{Int}(undef, num_participants, num_sections)
for i in 1:num_participants
    for j in 1:num_sections
        if rand() < 0.8 # 80% вероятности, что пойдет на секцию
            priorities[i, j] = rand(1:3)
        else
            priorities[i, j] = 10000
        end
    end
end

end

# Создание модели
model = Model(GLPK.Optimizer)

# Переменные решения:  $x[i, j] = 1$ , если участник  $i$  идет на секцию  $j$ 
model, x[1:num_participants, 1:num_sections, Bin]

# Целевая функция: минимизация суммы приоритетов
model, Min, sum(priorities[i, j * x[i, j]
    for i in 1:num_participants for j in 1:num_sections])

```

```

## Ограничения:

## 1. Каждый участник идет ровно на одну секцию
for i in 1:num_participants
    model, sum(x[i, j for j in 1:num_sections] == 1)
end

## 2. Ограничения по вместимости залов
for j in 1:num_sections
    model, sum(x[i, j for i in 1:num_participants] >= hall_capacities[j])
    model, sum(x[i, j for i in 1:num_participants] <= hall_max[j])
end

## 3. Особое требование для третьей секции
model, sum(x[i, 3 for i in 1:num_participants] == 220)

## Решение задачи
optimize!(model)

## Анализ результатов
println("Статус решения: ", termination_status(model))
println("Оптимальное значение целевой функции: ", objective_value(model))

## Распределение по секциям
distribution = zeros{Int, num_sections}
for j in 1:num_sections
    distribution[j] = sum(round{Int, value.(x[:, j])})
end

println("\nРаспределение участников по секциям:")

```

```

for j in 1:num_sections
    println("Секция $j: $(distribution[j]) человек")
end

println("\nПример рекомендаций для первых 5 участников:")
for i in 1:5
    assigned_section = findfirst(j -> value(x[i, j]) > 0.5, 1:num_sections)
    println("Участник $i: Секция $assigned_section (приоритет: $(priorities[i, assigned_section]))")
end

```

```

Статус решения: OPTIMAL
Оптимальное значение целевой функции: 1244.0

```

```

Распределение участников по секциям:

```

```

Секция 1: 180 человек

```

```

Секция 2: 190 человек

```

```

Секция 3: 220 человек

```

```

Секция 4: 200 человек

```

```

Секция 5: 210 человек

```

```

Пример рекомендаций для первых 5 участников:

```

```

Участник 1: Секция 4 (приоритет: 1)

```

```

Участник 2: Секция 4 (приоритет: 1)

```

```

Участник 3: Секция 1 (приоритет: 1)

```

```

Участник 4: Секция 4 (приоритет: 1)

```

```

Участник 5: Секция 1 (приоритет: 1)

```

Рис. 3.25: Решение задачи № 4

5. Кофейня готовит два вида кофе «Раф кофе» за 400 рублей и «Капучино» за 300. Чтобы сварить 1 чашку «Раф кофе» необходимо: 40 гр. зёрен, 140 гр. молока и 5 гр. ванильного сахара. Для того чтобы получить одну чашку «Капучино» необходимо потратить: 30 гр. зёрен, 120 гр. молока. На складе есть: 500 гр. зёрен, 2000 гр. молока и 40 гр. ванильного сахара.

Необходимо найти план варки кофе, обеспечивающий максимальную выручку от их реализации. При этом необходимо потратить весь ванильный сахар.

Для решения задачи используйте пакет JuMP и решатель GLPK (рис. 3.26):

```
# 5
# Создание модели
model = Model{GLPK.Optimizer}

# Определение переменных: количество чашек каждого вида кофе
@variable(model, x_raf >= 0) # Количество чашек "Раф кофе"
@variable(model, x_cap >= 0) # Количество чашек "Капучино"

# Цены на кофе
price_raf = 400 # рублей за чашку
price_cap = 300 # рублей за чашку

# Целевая функция: максимизация выручки
@objective(model, Max, price_raf * x_raf + price_cap * x_cap)

# Ограничения по ресурсам
# Зерна: 40г на Раф, 30г на Капучино, всего 500г
@constraint(model, 40x_raf + 30x_cap <= 500)

# Молоко: 140г на Раф, 120г на Капучино, всего 2000г
@constraint(model, 140x_raf + 120x_cap <= 2000)

# Ванильный сахар: 5г на Раф, всего 40г (нужно использовать весь)
@constraint(model, 5x_raf == 40)

# Решение задачи
optimize!(model)

# Вывод результатов
println("Статус решения: ", termination_status(model))
println("Оптимальное значение целевой функции (выручка): ", objective_value(model), " рублей")
println("\nОптимальный план производства:")
println("Количество 'Раф кофе': ", value(x_raf), " чашек")
println("Количество 'Капучино': ", value(x_cap), " чашек")

# Проверка использования ресурсов
println("\nИспользование ресурсов:")
println("Зерна: ", 40*value(x_raf) + 30*value(x_cap), " г из 500 г")
println("Молоко: ", 140*value(x_raf) + 120*value(x_cap), " г из 2000 г")
println("Ванильный сахар: ", 5*value(x_raf), " г из 40 г")

# Расчет выручки от каждого вида кофе
println("\nВыручка по видам кофе:")
println("От 'Раф кофе': ", price_raf * value(x_raf), " рублей")
println("От 'Капучино': ", price_cap * value(x_cap), " рублей")
```

Рис. 3.26: Решение задачи № 5

Статус решения: OPTIMAL
Оптимальное значение целевой функции (выручка): 5000.0 рублей

Оптимальный план производства:
Количество 'Раф кофе': 8.0 чашек
Количество 'Капучино': 6.0 чашек

Использование ресурсов:
Зерна: 500.0 г из 500 г
Молоко: 1840.0 г из 2000 г
Ванильный сахар: 40.0 г из 40 г

Выручка по видам кофе:
От 'Раф кофе': 3200.0 рублей
От 'Капучино': 1800.0 рублей

Рис. 3.27: Решение задачи № 5

4 Выводы

В результате выполнения данной лабораторной работы я освоила пакеты Julia для решения задач оптимизации.